Formal Architecture Guide for HLS Scheduling Rules

Constructive safety and required workload completion

Stuart Swan, Platform Architect, Siemens EDA

27 October 2026 • Aligned with the 27 October 2026 rules document

This guide explains how the current scheduling rules support source-level verification of an HLS implementation. The architecture has two principal results: constructive correspondence transfers observable safety from the configured source model to the RTL; required-workload completion adds a finite accepting source witness and implementation progress assumptions to prove eventual acceptance of the required outputs. The two results share semantics and evidence, but establish different guarantees.

This informative companion is aligned to the 27 October 2026 rules document, available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/catapult_user_view_scheduling_rules.pdf

The rules remain authoritative for definitions, theorem premises and applicability. Named references below identify their controlling interfaces. It is organized for architects, verification engineers and formal-methods implementers.

1 The architecture and its guarantees

The source reference is not automatically the original SystemC simulation. It is Sys_ref, configured with the implementation's relevant storage, channel boundaries, coupling and environment. The implementation is RTL_B. A theorem instance fixes these models, their initial and reset semantics, stimulus and choices, observation rules, and evidence bindings before correspondence is attempted.

For safety, Appendix J constructs a legal source execution prefix for every covered finite implementation prefix. The two prefixes have equal canonical observable histories. If the required property holds on every history in the independently declared source domain, it therefore holds on the covered implementation domain. This is the normal implementation-to-source direction: every covered RTL behavior is explained by a legal source behavior. It does not require the RTL to reproduce an arbitrarily selected source simulation.

For completion, Appendix K fixes a finite collection R of required testbench acceptances. A qualified finite source execution accepts R. Internal-fact correspondence and conditional agreement connect that witness to the RTL; controller, resource and service laws ensure the required operations actually occur. K.CT.Completion then establishes eventual acceptance of R with the witness's values and per-endpoint order for every qualified implementation execution satisfying those service assumptions. K.CT.LazySerial is the route name; K.CT.LazyComposition is its general mixed-component theorem, and K.CT.LazySerialRestricted is the restricted rank-derived corollary. K.CT.Completion remains the common final induction. The separate K.CT.LazyExtendedStandard corollary returns to Standard only under its restricted all-singleton construction and protected source-extension premises. The route comparison in §8 distinguishes these applications.

Result	Main inputs	What follows

Result	Main inputs	What follows
Constructive history coverage	Local source and implementation proofs, joint settling, environment, order and actual prefix carrier	Every covered RTL prefix has an equal reachable source history
Source safety transfer	History coverage and an independent property proof on the selected source domain	The extensional history property holds on the covered RTL domain
Internal-fact correspondence	Constructive local fact laws and exact coverage of the consumed fact signature	Completed internal and external facts match legal source facts
Required-workload completion	Full facts, agreement, a finite completed-unit witness/certificate, standard or lazy/composed exposure, and applicable progress/service laws	Required testbench acceptances eventually occur with matching values and endpoint order

Safety does not imply eventual output. Completion does not imply a cycle bound, a total order across unrelated endpoints, exclusion of extra outputs, or absence of every internal deadlock. Extra-output exclusion is a safety property. A deadlock after R has been accepted, or in unrelated activity, is outside K's conclusion. Source and implementation need not have equal registers, identical cycle counts or the same grouping of independent events into cycles.

The September architecture is no longer the application path. Appendix I is reserved. The former J.1/GWR certificate stack, KObs and wait-for-graph/Policy-S dominance argument are not the retained proof chain. M.7b, M.7c and M.8 are retired; there is no single-RTL-run B1 safety shortcut. B1 remains a separately claimable observable determinism property, not a substitute for constructive coverage, source safety or progress. Appendices R and S now describe the retained dependencies and mechanization targets.

2 Models, boundaries and observable histories

Selecting the source reference

Core.1 selects Sys_B for ordinary abstract channels. B(c)=0 means a primitive rendezvous: the matching Push and Pop commit together with the same value. B(c)>0 means a bounded, cycle-boundary, non-fall-through FIFO. A Pop needs an old item; a Push needs old free space. An initially full primitive FIFO cannot accept a Push merely because a Pop occurs on the same edge, and an initially empty one cannot satisfy a Pop with that edge's new Push.

Sys_B^elab(E) is used when a scalar depth does not faithfully describe the communication structure. It explicitly represents finite storage and the stateless deterministic couplers needed for staging, joins, bundles or synchronization gates. A nonempty or nonfull storage element may still be unavailable at a coupled process boundary: actual enablement uses the complete settled handshake equations. Storage or credit hidden from the model can change both safety and completion applicability. Exact capacity matching alone does not guarantee that storage releases available in one source schedule occur in a retimed implementation. LazySerial addresses that gap for eligible independent staging by relating each extra-slack release to a legally issued source consumer demand and its completed support.

Q.SafetyQualified(E) binds the elaborated state, accounting, ownership, effects, atomic groups, required order, frontier attribution and handshake network. It does not itself prove source/RTL correspondence, whole-system settling or progress. Q.CTQualified adds the applicable completion laws. Mathematical admissibility is also distinct from flow availability: a qualified model for which no executable or independently checkable source artifact is available cannot yet support the corresponding executable evidence workflow.

Keeping accounting and observation separate

The proof-internal alphabet Σ includes committed transfers at explicit modeled boundaries, selected synchronization and signal observations. An introduced staging transfer may be hidden from the DUT interface and still be a proof-internal event. Failed NB attempts emit no message event, but their outcomes and identities may matter to later behavior or to the completion fact signature. Request levels supply auxiliary Issue/attribution evidence at the selected evaluated boundary; they are not themselves committed message events. A step is ε only relative to the selected alphabet, so an outer-hidden transfer may still be a proof-internal event.

For elaboration, $\pi_{\Sigma,E}$ produces the complete outer accounting history, including both Push and Pop at every outer channel. Only afterward does ρ_{DUT} restrict that history to Σ_{DUT}. Thus $\pi_{DUT,E} = \rho_{DUT} \circ \pi_{\Sigma,E}$. FIFO balance is checked before DUT observation restriction; hiding a consumer event does not remove its accounting effect.

The state-computed signed map A_E must equal the channel-epoch Push-minus-Pop balance at the specified cycle-entry boundary. Its legal nonnegative range follows from that agreement and outer E4 legality, not from its signed type. Shared accounting views do not create extra storage or permit the same credit to be spent twice. Q.1 requires explicit ownership and invariant preservation for shared support.

J.OuterCanonHist records complete observable groups, occurrence identities, values and the required partial order. A rendezvous pair, paired synchronization, or qualified multi-effect atom is indivisible. Projection retains a complete selected group or erases it; it cannot retain an unauthorized half. Required paths through erased units survive as order between retained groups. Erasing a synchronization or reset therefore need not erase its ordering effect.

Three kinds of order

Source issue order, canonical observable order and proof-construction order serve different purposes. E3 constrains dynamic source-call Issue order and prefix closure, including across overlapped iterations. It does not automatically add a strict committed-event edge between distinct interfaces. The canonical order contains only the normative channel, synchronization, signal, reset and selected extension constraints. A descriptor ordering used to execute the proof is a legal witness serialization, not an additional observable dependency.

J.GeneratorCompleteness inventories every applicable ordering rule. J.RequiredOrderWF requires predecessor closure, stability of order between existing occurrences, and joint acyclicity. These are separate obligations: individually acyclic rule families can form a cycle when combined. Source primitive realization and a common rank can establish strictness; J.SourceOrderReflection can then transfer it to retained RTL groups before canonical equality is concluded. Any consumed erased-only graph still needs its own joint-acyclicity evidence under J.RequiredOrderWF.

A finite ideal of a canonical history retains whole groups and every required predecessor of each retained group. It need not be a prefix of one logger ordering, and it need not be operationally reachable. This distinction still matters to the selected-run source-property discharge in §6. It does not supply an implementation-prefix lift: that lift needs exact coverage under the selected Origin, Batch or separately proved extension route.

3 The operational contract behind the proof

Scheduling and request attribution

Appendix G's R1–R5 state source synchronization order, signal anchoring, isolation around synchronization, message issue order and channel semantics. E1–E5 preserve the corresponding implementation obligations, together with Core.17's occurrence and value matching. The compatibility predicate is ordered: its left argument is the source witness and its right argument is the implementation. It is not an unrestricted mathematical equivalence relation. R1 and E1 require strictly increasing cycles for distinct synchronization occurrences of one process. This does not impose equal source/RTL cycle numbers, order unrelated processes, or forbid a predecessor message from committing on its synchronization edge.

Issue is the first correctly attributed process-owned request assertion for the dynamic call. Commit is its process-facing completion under the declared boundary mapping. Core.10 and Core.IC require a fixed, bidirectionally complete mapping from requests to their actual owners. The endpoint-owned BoundaryReq at the selected evaluated boundary and its required stable payload or descriptor persist from blocking Issue until process-facing commit or an affecting reset; an arbitrary downstream physical level is not that attributed request. Back-to-back requests retain exact next-owner attribution. Physical capture, process-facing completion and later external delivery are distinct unless the mapping proves they coincide.

Stall gating has two sufficient bindings in Core.10. In the declared-complementary-gate form (GQ1), the operation-owned request persists and the gate is part of the actual complementary signal in E's coupler network. A different downstream signal view needs an exact transfer and occurrence correspondence, including effects, payload and storage. In the gate-before-issue form (GQ2), the gated own-side level is itself the request: gating may delay issue, but cannot withdraw a blocking request after issue except on an affecting reset. Neither form permits a fictitious signal mapping or proves staging progress by itself; an arrival-dependent gate cannot establish the pre-arrival request needed by the standard delivery proof. For example, GQ1 can retain an upstream request r while downstream valid = r∧g falls with g, provided the declared boundary refinement proves exact transfer correspondence. This is not withdrawal of the upstream BoundaryReq.

For distinct blocking calls on the same scalar explicit abstract boundary and endpoint direction within one reset epoch, Core.IC(4) and endpoint serialization give strictly increasing Issue cycles. Blocking Pops on different channels may issue in the same cycle: E3's distinct-interface constraint is non-strict. Distinct-interface no-reverse order and cross-interface issue-prefix closure still require their own evidence. A physical early read or movement inside a pipeline cannot be relabeled as a different source Issue merely to satisfy E3.

E5 also includes the completion fence: blocking calls in the preceding source interval must process-facing commit no later than their synchronization. Same-edge message and synchronization completion is permitted, but the canonical message-before-sync predecessor remains mandatory. New following-interval Issue is strictly later. An earlier buffered message may transfer externally later, because the fence concerns process-facing completion. A physical sub-cycle timestamp is not required to express that semantic precedence. The commit fence is a separate requirement, not a consequence of the Issue timestamp inequality alone.

Physical synchronization and direct-input updates

Core.SyncProtocol binds each sync occurrence to its actual participants, signal roles, clock edge and reset disposition. Once an endpoint offers ready or valid for that occurrence, it holds the offer and identity through commit or an affecting reset. An eligible settled ready/valid overlap commits the complete transaction exactly once; guards, matching and fence legality must already be established by the signals and supporting state. They cannot cancel the sampled handshake. A reset-only edge and any permitted transfer-before-reset

ordering are fixed in advance. A unary wait has its own declared release condition and completes at its first eligible release edge, without an invented peer or a later scheduler choice.

For sync-controlled direct inputs, the DUT must finish or protect every remaining old-value use before asserting ready. That protection continues while the sync is pending and for any later protected old-value use. The TB waits for ready, presents the new inputs with valid, and allows them to settle before the sampling edge. With ready held and no affecting reset, that edge completes the sync. The TB must not drop valid merely because a cycle elapsed without a handshake, and it keeps the input values stable until the next permitted update or affecting reset. Core.SyncProtocol supplies the formal lifecycle and pin-protection requirements; a fence or registered output alone does not establish them. A full pipeline drain is one sufficient way to protect old uses, not a requirement when their operands are retained and isolated. Core.16(7) separately requires withholding capacity belonging only to the next interval while the process is pending at sync; this mandatory admission control does not itself protect old-value users.

Preparation, settling and registered results

Core.Phase gives the source operational interpretation. In the L1 preparation window, a finite selection of legal evaluation and Issue steps may occur using already available operands. L2 freezes the relevant roots and settles the complete network. L3 realizes every physical message and sync completion required by that snapshot, together with any genuinely abstract actions permitted by their declared laws. L4 installs the registered results and simultaneous updates. Evaluation or Issue depending on a cycle-t return begins only at a strictly later Horizon under Core.RegSeq. A sync-induced gate change can change handshake eligibility only in a later cycle's L2 settling; it cannot change the already frozen cycle-t transfer inventory.

Core.SelectLatitude permits legal finite delay of L1 issue and only the latitude expressly declared for genuinely abstract internal movement or preparation. Prospective arbitration choices may affect the settled network. Once an eligible-edge physical handshake or unary sync release holds, its completion is mandatory. Nonselection cannot suppress that completion, withdraw an issued blocking request or change stable data. The issue policy, Core.16, synchronization, reset, atomicity, endpoint-cardinality and NB decision rules still apply. Complete-execution issue fairness, pre-grant arbitration fairness and internal service remain separate obligations within their stated scopes. K.CT.LazySerialWitness uses precisely this permitted abstract-movement latitude. K.CT.LazySimulation describes the executable source-side mode; it changes neither mandatory physical transfer equations nor the permissive reference relation used by J. Lazy movement is separate from source issue policy: Core.PolicyS or qualified group-serial issue governs calls, while lazy selection governs only optional abstract staging moves.

Core.TransferLaw applies to every declared physical ready/valid interface, including proof-internal transfers that are later hidden from observation. Ready and valid both high in the eligible settled sample means the declared transfer occurs on that edge. Old-state capacity, ownership, guards, reset priority and atomic coherence must be realized before overlap; an illegal effect is a conformance failure, not permission to ignore a handshake. The proof accounts for the complete forced transfer set exactly once. It may serialize those fixed effects for construction, but cannot add a transfer from intermediate updates or omit an inconvenient member. Statelessness alone establishes neither event identity nor a single atomic transaction.

The construction must prove those source steps legal under Core.ReachPrep, Core.SourceChoiceBinding and Core.PreparationEvidence. Constructor success is not the definition of legal preparation. An NB result follows Core.NBDecision from the actual qualified snapshot and fixed resolutions; it cannot be chosen by an annotation or manufactured by declining a transfer that the decision law requires.

J.SourceSettleRealization supplies a finite legal path to the first operational L2 fixed point and matching boundary values. The whole combined graph must include all relevant requests and absences, nonminimal

pending calls, NB attempts, environment logic, guards and frontier predicates. Qualifying each coupler separately is insufficient when cross-component dependencies have not been checked.

Ordinary locality offers two source-settling paths: QF reuses a qualified source QSync result; QSafe supplies the narrower safety-only operational evidence. Both retain finite deterministic acyclic evaluation and require QM source/implementation snapshot matching. Separate QSync qualifications do not themselves prove matching. QSync also does not prove boundary-update commutation, process refinement or Batch prefix semantics. The retained sufficient constructive profiles do not acquire combinational-feedback support merely because a broader fixed-point argument might be conceivable.

Blocking does not freeze all physical computation

Core.16 activates at the first settled snapshot where the nonempty pending blocking frontier is fully disabled. It does not retroactively invalidate legal eager issues from the preceding L1 window. While that activation persists, it forbids new source-later launch, including source evaluation and ReachPrep as well as Issue, but permits qualified progress of already-admitted work. That progress has two distinct cases: downstream drainage of already-admitted work, and completion of a source-later request legally issued before activation. Source order is not inferred from physical stage location. Under the Core.16(5) clarification, completion of pre-issued younger work must preserve the values and ownership of every live operand, pending request descriptor, attributed storage and pending effect still needed by stalled older work. Physically disjoint stage registers are one sufficient realization; banked, tagged or other shared storage needs a proved ownership/noninterference invariant. Permission to commit is not permission to overwrite live state.

Physical private computation may continue if it is abstractly invisible: it changes neither represented source progress nor protected requests, payloads, storage, readiness or admitted work. Later use must match the legally evaluated source operands and their writer/occurrence identities, respect registered-return timing, and remain legal under any persisting activation. Computing privately from a FIFO head does not authorize a source computation or dependent request before the corresponding Pop’s legal return use.

Core.16 and its finite decreasing drain measure constrain permitted behavior. They do not force movement. B.OF and B.CT separately supply the positive implementation laws needed for completion: eventual request assertion, finite serviced computation, ordered movement and useful downstream drainage.

4 How constructive correspondence is established

The local construction

J.ConstructiveEdgeInduction is the shared induction principle. From every prescribed implementation base, construct a legal related source base. For every construction-reachable related origin and every actual implementation successor, extend that exact source prefix legally, retain its previous choices and identities, restore the relation, and establish equal extended canonical histories. Induction over actual finite edge ancestry gives origin coverage. No fair extension or future cycle is assumed; deadlocked and terminal finite prefixes remain in the domain.

Core.ConstructiveOriginCarrier fixes actual origin occurrences and their finite edge ancestry independently of correspondence success. A non-identity designation requires an occurrence-level transport proof preserving the represented state, roots and choices and covering every actual continuation through the declared continuation or commuting-step correspondence. Equal histories alone do not establish that transport, and no future input, edge or fairness may be assumed to obtain it.

For the registered message profile, J.LocalOriginContracts C0–C5 discharge that induction through the following evidence.

Contract	Information that crosses the boundary
C0 Base and environment	Legal matching bases and a source counterpart for each actual environment step under the fixed choices
C1 Roots and attribution	Complete requests and absences, operands, dynamic owners, NB attempts and correct frontier activation
C2 Settling and complete action inventory	A legal matched snapshot, every forced physical completion and NB outcome, and any permitted abstract actions
C3 Boundary realization	Primitive legality, complete mandatory effects, resource residues, atomic groups and frames that compose into one legal source segment
C4 Update and preparation	Simultaneous registered update and finite legal owned preparation restoring the relation
C5 Canonical translation	Complete group matching and both directions of normative generator/path translation

C3 is where separate local legality becomes joint executability. A finite acyclic descriptor graph orders the construction. Each component describes its residue after any predecessor-closed subset of completed descriptors. For a minimal remaining descriptor, its participants establish the actual source primitive's premises and exact effects; unaffected components are framed. Shared writes need one joint owner or a proved combined update. The inventory includes every transfer forced by the common snapshot, with exact identity and compound membership. This prevents both mutually inconsistent local witnesses and a selectively incomplete transfer set from being treated as a global execution.

C4 restores the relation using the actual source control and return rules. Prepared local words compose because their writes are owned and their operands do not depend on another process's newly prepared requests. A terminal export may end at its completed L4 checkpoint without inventing a next L1 window. If a real later successor exists, its case must still establish the legal phase entry it consumes.

C5 proves equality of the semantic histories, not just equal event lists. It preserves whole groups, exact identities and values, old history restrictions and all required paths in both directions. J.LocalBoundaryCompose, J.LocalOriginCanonExt and the update/preparation proof together establish J.LG-Origin. They supply no progress conclusion.

From origins to all finite prefixes

Core.RegisteredMessageBatch is an explicit implementation completion profile, not a consequence of calling a design synchronous. All mandatory physical completions, declared abstract completions, NB retirements/results and the simultaneous update at an edge become complete together. Between edges, silent activity does not change the selected canonical history. The initial state and every actual completed edge, including a terminal edge, have a qualified origin.

J.BatchPrefixCoverage therefore associates an arbitrary finite prefix with the initial history or its last already completed edge. It does not finish the current cycle, require future input, or select a fair continuation. Combining this lift with J.LG-Origin gives exact all-finite-prefix history coverage for the declared Batch domain. Co-completion does not merge independent groups into one atom or add order between them.

A monitor using this guarantee assembles the complete edge bucket before evaluating a transferred property. Its result must still be extensional on the canonical history: changing erased timing or incidental listing order cannot change the verdict. A cycle-count bound is not made extensional by batching. A finer implementation carrier or monitor exposing partial inventories needs its own exact correspondence proof.

The cycle-ideal lift, including J.ConstructiveOriginIdealSafe, J.CycleIdealCover and its exclusive partial-cycle/carrier machinery, has been removed. Core.ConstructiveOriginCarrier and J.LG-Origin remain independent of that removed route; exact all-finite-prefix coverage uses the declared Batch lift or a separately proved exact-history extension. Anchor, reset, configuration and R2C profiles each retain their own lift. No route may silently narrow an already declared implementation domain. J.IdealPrefixClosed remains relevant to the independent selected-source-history property method in §6, not as a replacement for exact implementation-prefix coverage.

Locality theorems and their limits

J.OrdinaryConstructiveComposition provides a stronger organizational result for the prospectively defined OM and OA profiles. It assembles the global relation from process-local LP0–LP5 evidence, ordinary primitive refinements, G1–G7 integration facts, settling/matching and environment/carrier evidence. Within that scope, no extra application-specific global correspondence invariant is an input. Local and integration evidence must account for every physical completion forced by the common settled sample, as well as its legal effects and updates. Local evidence is substantive and universally quantified over the admitted incident interfaces; it cannot hide a peer’s private state, future response or the desired global continuation.

OM has independent ordinary FIFO/rendezvous resources and message-only observation. OA adds supported unary waits, paired synchronization and qualified sequential signals with pre-update sampling. These locality profiles exclude shared capacity and endpoint ownership, shared mutable application memory, dynamic reset, general configuration/R2C extensions, unsupported forwarding/resolution and selected START/FINISH conventions. Such features are not declared universally illegal; they require a broader qualified construction. Where a profile does retain FINISH_P, it is the distinct event emitted by actual termination in the epoch, not a retrospective label for the last event of a finite prefix.

J.ElaboratedConstructiveComposition gives ELM/ELA locality for separately owned FIFOs and the specifically qualified MoveAB, request-fed JoinFire and FIFO-fed ForkABC schemas, while retaining the other ordinary restrictions. It requires Q.SafetyQualified, expanded local packet/bookkeeping evidence, complete NB decision binding, a closed generator inventory and complete forced-transfer inventories under the actual compound equations. It is not a theorem for arbitrary elaboration, shared pools or zero-storage compounds. An internal move with no process completion still needs its register-update and local-preparation proof. The Q.CT staging and pair profiles do not add atomic process-facing paired-Pop retirements, internal joint moves consuming one old head from each of two ingress FIFOs, or simultaneous-replacement staging shifts to ELM/ELA; those use the general constructive route with complete C0–C5/A0–A6 evidence. For replacement-capable staging, Q.CT.InputStagingResidue states the fixed-selection residue obligation that the C3 boundary realization must discharge, under its own M.7a entry.

These are conditional paper-level composition results. The rules do not assert that every R/E-correct implementation admits the required local evidence, or that a concrete HLS flow, automatic extractor or mechanization has already been qualified.

5 Selecting observations and feature extensions

The general constructive architecture has named extensions beyond the ordinary locality theorems. Select the actual feature and observation profile before proving coverage; do not infer a combined result by conjoining independently proved relations with incompatible roots or state.

Selected feature	Controlling interface or theorem	Essential qualification
Registered messages	Core.RegisteredMessageBatch; J.LG-Batch-Safe	C0–C5 and actual whole-edge completion semantics
Stable direct inputs with message observation	J.StableDirectInputContracts; J.StableRead-Batch	Real initialization anchors, stable values, in-place substitution and legal source lifting
Waits, paired synchronization and signals	Core.RegisteredAnchorBatch; J.LG-AnchorBatch	Unary waits and pairwise SyncChannel/ac_sync only; barriers composed from these need joint evidence. Core.SyncProtocol, A0–A6, complete anchor membership, samples, last-Write fold and joint update
Direct inputs with retained Reads	J.AnchorDirectInputLift	Anchor-determined Read descriptors retained in whole groups; per-interval value binding and old-work isolation
Transformed internal input payloads	Core.LogicalPopPayload; Q.CT.LogicalPayload and LogicalPair; J.LogicalPayloadRefinement	A local discharge inside the selected route, not a separate coverage theorem. Causal ownership, physical representation and legal later source use; excludes NB/signal/direct-input observations, data-dependent routing, side exits, affecting reset and epoch gates
Configuration records	Core.AnchorConfigObservation; J.LG-ConfigBatch	CO and CVG contracts for each governing record and every actual use
Dynamic edge-sampled reset	Core.RegisteredResetBatch; J.LG-ResetBatch	RB0–RB5, scoped disposition, epoch identities, hold/release and initialization
Edge-observed combinational signals	Core.RegisteredR2CBatch; J.LG-R2CBatch	RC0–RC6, joint acyclic settling and exact emission/participation

For ordinary retained sequential Reads, RULE 1 fixes membership and value at the preceding anchor; a later message-dependent branch cannot retroactively add a Read to a completed anchor group. Multiple Writes sharing a succeeding anchor are folded in executed source order, with the final Write supplying the retained value. Physical samples do not create extra logical Reads or move their anchors.

J.AnchorDirectInputLift supplies the retained-Read option when direct inputs coexist with observed anchors. It substitutes the legally governing value in place within each interval while keeping the complete anchor-

determined Read descriptor in the synchronization group. Every physical sample and old-work operand must meet the stability/update-isolation contract, including Core.SyncProtocol's protection before a ready offer authorizes pin changes and throughout all remaining old-value uses. Conditional or repeated evaluation is allowed only with the qualified anchor-determined membership; value equality alone cannot establish that membership or justify deleting a Read from a retained group.

StableRead handles a narrower message-only case by substituting a fixed configuration value at each reached read expression and lifting the reduced proof back to the original source. It does not hoist conditional evaluation or waive Core.16. ConfigBatch instead records designated configuration values at real initial anchors or governing paired transactions. Designated uses create no observable process-Read labels in that option. Every actual use must still receive its governing value, and old work must be isolated from newly presented pins. Zero-length intervals and conditionally unused fields still have their declared configuration records.

Reset matching preserves actual scope, priority, cancellation and storage effects. Discarded contents are not fabricated Pops; balances restart only for affected channels. Partial reset across live communication requires a modeled cross-epoch disposition. History coverage does not require eventual reset release. Asynchronous assertion needs its separate implementation argument, and safety across reset does not prove preservation of outstanding K workloads. The empty-channel initialization convention applies to the bound initialized channels; a dynamic reset empties only its affecting channel scope and preserves unaffected contents under RW5.

R2C observation is of the qualified fixed point, not transient delta-cycle propagation. Ordinary emission uses participation plus first observation, changed value or a fresh sampler; cycle-locked emission requires a unit in every participating cycle. The reset ledger restarts the appropriate comparison state without renumbering prior history. A repeated numerical value does not excuse an omitted required unit. The combined constructive profile requires compatible local relations, inventories and the complete acyclic settling graph.

For transformed internal input paths, Core.LogicalPopPayload and the representation and operational clauses of Q.CT.LogicalPayload permit causal original-value attribution at the actual logical Pop retirement without requiring raw-value recovery from current registers. J.LogicalPayloadRefinement supplies the corresponding local safety cases without completion obligations; it is a local discharge inside the selected constructive route, not a separate coverage theorem. The proof still needs actual admission/ownership, physical representation, legal later source use and all remaining constructive cases. Physically sampled observations are excluded; projection cannot relabel them. The attribution creates no transfer, changes no readiness, retimes no event and licenses no early source use. The profile covers blocking singleton Pops on positive channels and the declared Q.CT.LogicalPair; it excludes NB, signal and direct-input observations, data-dependent routing, side exits, affecting reset, epoch withdrawal, cancellation and rendezvous groups, and it does not enlarge ELM/ELA coverage. Q.CT.LogicalPair and, where selected, Q.CT.PairedRepresentation preserve both logical owners and one complete atomic return packet for a computed pair. Reusing a property proved on an ungrouped source requires a separate group-erasure/inclusion proof covering observations, required order and admissible cuts; a simultaneous physical handshake of two Pops does not supply that proof.

In the single-shared-slot realization of the reserved-empty LogicalPair profile, a pair issues only when its process-owned result slot U is empty. If that slot remains reserved through pair retirement and occupied until the dependent output Push commits or a later-edge safe advance releases it, the following timing limit applies. That reservation is a genuine issue prerequisite: the next pair cannot issue in the L1 window before the releasing edge, so the sustained pair initiation interval is at least two cycles. A serial U3-to-U4 split with the transfer at the later releasing edge does not remove this limit: U3 is still occupied in the L1 window before that edge. A two-slot reservation scheme needs its own explicit slot-ownership, reuse, ordering and backpressure

evidence, and allowing issue against an occupied slot is a different handshake profile under Q.CT.Extension. In every case the completion theorem guarantees no rate.

The current shared-memory safety route needs J.Mem lowering to explicit value-labeled channel or anchored-signal operations. A general memory-state extension is discussed but is not supplied by the current theorem; using one would require a separate proof. Synchronization alone establishes neither read-value equality nor that lowering. Transaction-changing mappings additionally use Core.MemProfile/Core.MemFaithful. Core.MemSyncOrder remains a separate scheduling-conformance obligation; it does not add raw memory events to the constructive alphabet. Completion must expose hidden memory backpressure and service conditions. Multiple clocks remain outside the single-clock theorem instance; Appendix O is an informative sketch, not a CDC composition theorem.

6 Source properties, testbenches and stimulus classes

What the source verification must establish

J.ReachableHistCov uses the fixed outer history CanonHist_Σ_DUT: every implementation prefix κ in the declared domain D has a legal reachable source prefix λ with equal CanonHist_Σ_DUT histories. The configuration route applies the same semantic coverage condition to its route-local history CanonHist_cfg (J.LG-ConfigBatch). J.SafeTransfer-HistCov transfers an extensional predicate Φ once Φ is independently established on every reachable source history. The generic-H interface is J.ReachableHistCovOn(D,A): H and the source domain A are independently bound, and every κ in D has λ in A with H(κ)=H(λ). J.SafeTransfer-HistCovOn supplies its property transfer. The correspondence proof does not establish functional correctness of the source design.

Core.SourceCheckpointProfile can narrow the source-property domain to independently declared initialization or completed-cycle checkpoints ChkRef. J.SourceCheckpointRepresentative proves that constructed witnesses have equal-history representatives there; the actual constructed prefix remains the inductive state. J.SafeTransfer-HistCovOn then transfers a property proved throughout that domain. Ordinary cycle-granular simulation alone does not prove the representative theorem or cover all unresolved source choices.

A second property-discharge option uses one selected complete source execution ρ^o. It needs J.RefProjCover, or RefProjCoverOn for the declared source domain: every relevant source history must be a finite ideal of $H(\rho^o)$. It also needs Φ on every finite canonical ideal of $H(\rho^o)$. Checking ordinary prefixes of that one run is insufficient. A directly proved all-ideals premise requires no ideal-closure assumption; deriving it from one successful finite-history evaluation does require the separate closure argument. Restricting the covering domain does not weaken the all-ideals property premise of this transfer clause.

For example, a scoreboard can evaluate an occurrence-keyed request/response ledger after merging a complete edge bucket. It must express matching and values from H and be invariant under every timing/order distinction that H erases. If a needed relation cannot be recovered from H, select and qualify a richer observation or use another property theorem. A logger cannot invent a canonical dependency to make the property transferable.

Reusable stimulus-class correspondence

J.ProcessEnvClass fixes a family of stimuli, initial data and choice resolutions. The primary profile includes the testbench processes in both compared systems; the implementation-side testbench need not be synthesized, but its runtime, libraries, wrappers and interface semantics must be qualified. Reactive responses follow from related testbench state and actual return values. Identical testbench source code alone does not establish that correspondence. Identical random seeds do not suffice either: latency-dependent timeouts, polling or clocked

reactions can generate different later inputs. Replication, prerecorded stimulus and recorded replay each need the selected stimulus/choice/environment binding; trace-functional reactions alone prove neither legal replay nor correspondence.

An external-testbench profile is also available when its explicit environment relation supplies legal source counterparts for every actual environment step and consumed observation. It is not automatically a cheaper or weaker contract. Neither placement permits substitution of a more cooperative environment or reclassification of reactive responses as freely reselectable stimulus.

J.UniformConstructiveEvidence organizes class-wide obligations: UCL0 binds the family, instance semantics, history and source-property domain; UCL1 supplies uniform local construction over every actual successor; UCL2 establishes environment/testbench composition; UCL3 supplies complete prefix and projection coverage. Class membership must be independent of constructor success. The named locality theorems can discharge their portion of these obligations within scope.

J.UniformConstructiveCoverage then gives history coverage for every admitted stimulus and finite prefix. J.ConstructiveClassSafe adds the independently proved source property throughout the class. Applying that result to a member needs exact binding and residual membership/applicability checks, rather than a fresh mathematical correspondence proof. Sample campaigns, a reusable structural artifact or one testbench program do not establish these universal premises. A completion-class claim separately binds and proves its workload, fact, agreement and progress obligations.

7 The bridge from safety to completion

Completed facts rather than outputs alone

K consumes J.InternalFactCorr over an explicit signature F. F includes R's acceptance facts, every workload unit's member completions, and every historical control/value predecessor used to identify those operations and any recorded demands consumed by the lazy route, recursively. For the standard route it also includes every historical fact recursively supporting K.CT.CtrlSupport decisions, including decisions that a conditional operation is absent; a negative decision is not itself a fabricated retirement fact. It includes failed NB outcomes or other non-transfer observations whenever the completion proof uses them. Initial facts are bound separately. A recorded demand is not a completion fact. Its alignment, preparation and control-support facts belong in F when consumed; actual Pop completions added by protected source extension also belong in F. Facts coverage remains every finite prefix of every operationally admitted implementation execution, independently of service and success.

J.ConstructiveFactEvidence adds CF0–CF3 to the selected constructive route: matching base facts and stable identities; an exhaustive actual-edge inventory; legal local source realization and preservation of all old facts; and an exact finite-prefix lift for the entire F ledger. Core.RegisteredFactBatch and Core.RegisteredFactPrefix provide that lift in the qualified registered specialization. Outer message Batch coverage alone does not prove that all internal results have the same prefix semantics.

J.ConstructiveFactCorr derives J.InternalFactCorr; J.RegisteredConstructiveFactCorr specializes it for qualified message/anchor constructions. Two named adapters cover configuration-bearing inputs. J.StableReadFactCorr lifts the full fact signature through the stable-direct-input reduction, binding θ as a common initial root. J.ConfigRecordFactCorr covers committed CO1/CO2 configuration records and their governing-use attribution; when K consumes a post-base record, its synchronization commit is a W unit preceding the dependent target. Neither adapter follows from the corresponding message-safety theorem: both require complete local fact evidence and exact full-F prefix coverage under CF0–CF3. ConfigRecord uses FB0–FB2 for its registered carrier; StableRead permits that discharge or CF3's separately proved exact partial-prefix lift. Other extensions need

their own full fact cases. Fact correspondence establishes what completed, with what identity, value and order. It does not establish current request presence, microstate equality, enablement or fairness. Those remain explicit conformance invariants.

Agreement is conditional on occurrence

Core.FinitePrefixProvenance gives a common fact-key universe, legally fixed roots, deterministic typed evaluators, a fixed finite predecessor set for each non-root fact, and a common well-founded predecessor relation. Its independently declared domain includes both the workload witness and all source prefixes supplied by internal-fact correspondence. Occurring facts must be available with their historical values, and all evaluator predecessors must be legally available; future results and retroactively reselected observations are forbidden. If a source witness is extended after R to complete selected singleton demands, Agreement covers the witness actually used, including those added steps. The extension may enlarge F, but does not change the implementation-prefix domain of Facts. The selected J adapter must cover the consumed signature; determined values alone do not establish coverage.

Core.FixedSupportValidation explains data-selected dependencies without changing that fixed-support theorem. A conditional read or loop decision may determine which earlier fact is consumed. The witness proposes support; the certificate must first validate the lower-ranked selectors and their agreement across the independently fixed domain, then establish actual availability and attribution of the selected facts. It cannot use the agreement theorem on an unvalidated certificate, assume future facts, or require unexecuted alternatives to be available. If no common fixed support can be proved, use K.CT.Agreement’s independently proved alternative; one successful witness is not the validation.

Core.FinitePrefixAgreement proves that the same available fact has the same value in two such prefixes. J.InternalFactAgreement transports that conclusion to matching RTL facts. This says nothing about whether a missing operation ever occurs. An evaluator for an operation also determines whether its descriptor exists, including its control/iteration decision; agreement is not merely payload equality after assuming the desired operation exists.

The default serial workflow consequently needs common timing-independent provenance for consumed control and values, or a separately proved observation/outcome alignment with equivalent coverage. An NB arbitration winner or signal sample that varies with issue timing can invalidate that agreement even if each execution is individually legal and safe. A deterministic simulator, B1, or one accepting source run does not repair the missing premise.

The logical-payload safety profile described in §5 can also supply facts to this completion bridge, but only with the complete CF0–CF3 and applicable fact-carrier cases. Its logical values are fixed causally before comparison with the workload witness. Representation alone supplies neither agreement nor a legal witness lift, exposure, delivery or service; all separately triggered K.CT obligations remain.

8 Required-workload completion

The finite witness and prerequisite graph

K.CT.Instance fixes finite required testbench acceptances R prospectively, with their logical keys, values and per-endpoint order. Acceptance is an observed receiving event, not merely output assertion or a DUT-side write. The clock, local updates and required environment service continue until acceptance. No affecting reset, cancellation or premature termination may remove an outstanding required obligation without a separate preservation qualification.

“Qualified implementation execution” means an operationally admitted execution of the prospectively bound instance satisfying its service and other execution-level contracts, not an execution preselected for success. Facts covers every finite prefix of every operationally admitted execution independently of service, completion or constructor success. Exposure covers its service-qualified domain. Standard supplies a conditional exposure proof; it does not narrow the Facts domain to successful runs.

K.CT.Witness supplies a finite legal execution of the same Sys_ref accepting R and a finite set W of commit units containing R and its prerequisites. Every W member actually occurs in that witness. Units and finite predecessor sets are fixed before the implementation execution being proved, form an acyclic graph, and retain all consumed resource/control dependencies or their separately proved invariants. Completed obligations remain completed in the bound epoch.

For the standard route, each target q in W also carries K.CT.CtrlSupport(q): the finite, provenance-keyed entry, branch and membership decisions needed for placement and preparation. They cover the required blocking I/O of older iterations, the target iteration’s entry permission and the membership of its preceding stages, or a sequential owner’s earlier control path. Negative decisions, such as an untaken conditional site, are recorded with their evaluator, witness result and recursive support. These are control records, not commit units: W still contains only actual witness completions, and absence of a site from W is not evidence that it is untaken. A general or extension exposure proof may use another schema but must discharge the same control, entry and membership dependencies it consumes. If no finite, complete, acyclic certificate exists, the standard route is inconclusive. The same prospectively bound CtrlSupport schema applies to legally recorded lazy demands using Alignment and DemandPlacement or BatchPlacement. Those demands remain auxiliary records until an actual completion is included in W. Source issue evidence does not assert RTL request presence; the placement and preparation proof derives that request.

The default K.CT.SerialWitness uses Core.PolicyS: within one synchronization interval, an owner does not issue a later blocking Push/Pop until every earlier source-order blocking Push/Pop in that interval has committed in a strictly earlier cycle. Independent owners remain concurrent. The source model uses the bound capacities and real enabling laws, or a separately proved smaller-capacity lift. The witness stops at a finite prefix satisfying Core.RequiredAcceptanceObserved; it need not drain every FIFO, finish trailing loop-head Pops or satisfy a maximal-run response monitor. Where eligibility holds, prefer running this default workflow in lazy mode. Plain unreserved FIFOs require no additional movement restriction; within the shared restricted profile, their resource proof reduces to the ordinary floor argument. Atomic groups still require the separate group-serial discipline. Using lazy selection at unchanged capacities does not itself require a capacity lift or a port-buffer-removal setting.

A primitive rendezvous is one unit containing both endpoints. Genuine atomic groups remain indivisible. When same-owner atomic members cannot be issued under ordinary Policy S, K.CT.GroupSerialWitness uses a qualified group-serial restriction of Policy L: all independently prepared members issue legally in one pre-settle window, while every preceding group required by each participating owner’s source order has committed in a strictly earlier cycle. Groups cannot cross synchronization or reset boundaries. This is not a redefinition of Policy S and does not remove unresolved sibling dependencies. Qualified overlapped witnesses retain their own preparation and residual obligations. Joint support and any pre-grant or abstract/internal service need their own qualification; an eligible physical joint handshake then commits indivisibly without a second fair-selection step.

K.CT.Closure is stronger than collecting dependencies visible in one favorable schedule. For the standard profile it includes lower-ordinal endpoint operations; source-earlier required blocking calls on every owner’s control path; all required I/O in preceding stages and older iterations of a pipelined owner; matching inputs; guaranteed-floor space releases; and all consumed control/value and sync/gate predecessors. Every non-root

fact supporting a CtrlSupport decision is in F and supplied by a W unit strictly preceding the target, or the decision is discharged by an independently proved invariant that does not assume the placement or completion it supports. This is what establishes that the listed earlier work omits no required blocking operation. Partial-stage sibling omissions need the explicit legal pre-activation preparation qualification. These dependencies organize a sufficient proof; they are not commands forcing RTL to execute serially. This owner-path requirement applies to mixed processes as well: include required prologue operations, preceding loops and their iterations, and loop work before an epilogue target in the current synchronization interval, subject to the stated qualified batch exceptions. A lazy demand certificate replaces only specifically justified extra-slack floor edges; it does not claim that their original retirement predecessors completed. All other consumed dependencies remain. The resulting completed-unit graph satisfies K.CT.Witness and obtains Exposure from the selected local proofs.

The preferred lazy pre-HLS workflow

K.CT.LazySimulation makes the simulator change explicit. For each optional abstract internal move of item j, retain the normal item, destination, ordering and edge conditions, and additionally require that the matching process-facing Pop(j) has issued and remains unretired in the same pre-HLS execution. Apply the condition to each staging step, including the ingress-to-staging move (early capture in the B.BoundaryBinding sense). It prevents prefetch before demand issue but permits release of ingress space before Pop retirement. It does not change source Push/Pop calls, capacities, the selected serial/group-serial discipline or physical handshake laws.

The relevant Pop is the source operation that returns the item to the consumer. An earlier read at the pipeline’s outer interface can instead be movement into channel storage. The fixed boundary map distinguishes them; outer ready alone does not identify process-facing demand. The channel model or simulator scheduler uses its own source-demand record. The producer does nothing differently: it asserts its legally prepared request/valid without adding a wait for ready and retains the request and payload while blocked.

K.CT.LazySimulation.DemandVisibility requires the recommended simulator mode to expose a legally reached and prepared Pop request independently of matching-item availability, once the selected issue discipline and phase rules permit issue. The requirement applies to the complete legally prepared batch for an atomic group. A read implementation that records demand only after data arrives can stall the movement needed to deliver that data. This requirement does not bypass Core.16, Core.RegSeq or other issue restrictions, and demand need not strictly precede arrival when data is already available.

K.CT.LazySimulation.Selection recommends prompt deterministic selection of jointly legal moves, respecting old-state tests, simultaneous updates and conflicts. An item cannot cross multiple locations on an edge contrary to the component law. Later selection remains legal where the reference permits it; the rules prove no universal dominance of prompt scheduling. Mandatory physical handshakes cannot be declined. At-site split capture into attributed slots is a separate reserved-slot realization and is outside the current lazy profile.

The added mode uses only the pre-HLS run. It does not query RTL requests, RTL progress or RTL completion for each stimulus. Generated structure supplies capacities, topology and boundary mapping; reusable implementation-family conformance and instance applicability supply the operational guarantees. The proof then derives corresponding RTL demand and release progress. If another selected qualification independently requires RTL snooping or co-simulation for witness-significant outcomes, that dependency remains. LazySerial neither requires nor removes it; see K.CT.LazySimulation.RTLIndependence.

Keep three simulation choices distinct: the source issue discipline, the optional movement selector, and the actual capacity/boundary configuration. A declared physical skidbuffer handshake remains mandatory. For

throughput or timing analysis, use a separately qualified model for the selected configuration; neither an accepting lazy completion run nor its certificate establishes a sustained rate or universal source safety.

Example with one ingress entry and two staging locations

Assume an eligible independent input path with F=1, m=2 and no outstanding-credit cap. Item a is in ingress, staging is empty and the consumer has not issued Pop(a). Normal mode may move a into staging if its ordinary conditions permit. Lazy mode holds a in ingress. A producer offering b remains pending with its normal valid behavior. Once the pre-HLS consumer legally issues Pop(a), a may advance; freeing ingress allows b on a subsequent eligible edge even before Pop(a) retires.

Item b cannot enter staging until Pop(b) issues, which singleton Policy S forbids before Pop(a) retires. Thus the lazy witness holds at most F+1=2 unmatched items here, while physical modeled storage remains F+m=3. The restriction limits use, not capacity. Full use requires a separately qualified movement/progress profile, such as an elastic-staging extension, not supplied by this example. For a qualified atomic input group, legal batch issue permits each admitted independent member item to advance without awaiting the other inputs; retirement remains atomic.

Empty staging is not automatically usable slack. If a producer sends more items than ingress can hold and only then sends a start message, while the consumer waits for start before issuing any data Pop, lazy mode cannot move the first item out of ingress. The source run stalls despite empty staging. This is inconclusive about that RTL implementation; another qualified witness or stronger forwarding profile may succeed.

Atomic-batch example. Let c have one ingress entry, two optional independent staging locations and no credit cap (or C>1). Let e be an unreserved positive FIFO with space for x. The producer executes c.Push(a); c.Push(b); e.Push(x). The consumer legally prepares and issues one same-site atomic batch {c.Pop(), e.Pop()} independently of item arrival. The whole group meets LazyEligibility and BatchPlacement, including coincident retirement without an attributed slot for the FIFO member.

Batch issue lets a advance on c without waiting for x. After that ingress release takes effect, b can enter on a later eligible old-state test; the producer can then send x. Once both inputs and the joint guard/service conditions are satisfied, the batch retires a and x together. Item b waits in ingress for its own later demand. Registered-return timing and the actual movement laws still govern each step.

With only a depth-one FIFO on c, a cannot retire before x arrives, b cannot enter before a retires, and the producer cannot reach the send of x. The staged design’s standard floor-only certificate has the corresponding cycle; qualified member-demand support removes that particular cycle without assuming batch retirement. Complete graph and conformance checks remain necessary. Separate sequential Pops would change the example. Lazy staging therefore is not generally equivalent to removing elaborated structure; any such replacement needs its own qualification.

How the witness supplies release evidence

K.CT.LazyInventory establishes at most one unretired staged item per selected channel, belonging to its outstanding demand. Before witnessed Push(k), let r be the number of retired matching Pops. With k earlier Pushes, unmatched inventory is A=k−r. The old-state ingress-space test and the lazy invariant imply A≤F. With g=min(F,C), or F without credit, ordinary cases use the standard floor resource proof, including the initial k<g case. Extra slack means A=F: the sole staged item is r=k−F, its Pop has issued but not retired, and credit is absent or C>F. If C≤F there is no extra-slack case. These are K.CT.LazyInventory and LazySlack.

K.CT.LazyClosure recursively closes the restricted singleton witness’s dependencies over a finite universe: completed units plus issued pending singleton Pops whose matching Push has committed. A supporting

completed Push can itself introduce another pending floor demand. The set is fixed before extension; this is not a depth-one argument. Policy S supplies at most one outstanding blocking operation per owner. Pending nodes enter through extra-slack floor edges, not as completed control facts. They are not W members merely because they are recorded.

K.CT.LazyRank assigns Pushes their commit times and singleton Pops the later of issue and matching-admission time, with Push before Pop on a tie. K.CT.LazyMixedRank combines those modified singleton ranks with actual group-commit ranks and member-demand ranks based on batch issue and matching admission. The serial/group-serial, source-order, old-state and complete support hypotheses make all relevant edges increase rank. This derives acyclicity in the restricted profile; it does not waive validation of eligibility, witness membership or dependency completeness.

Certificate branches and the theorem hierarchy

The singleton extension branch uses K.CT.LazySourceExtension for a fixed finite set D of issued, unretired eligible singleton demands, each with an admitted matching item, retired lower-ordinal Pops and completed preparation support. Legal optional moves carry each oldest item through empty forward locations to terminal retirement under the reference transfer or finite guard-resolution rule. The bound testbench must permit continued clocks and a whole-design window without affecting reset, cancellation or termination until D completes. Restricted LazyClosure supplies the local premises; mixed use establishes them directly. A pending group or missing sibling admission is not supplied by this lemma.

Actual extension completions may be added to W and F. Agreement covers the extended witness and the fact adapter covers the enlarged signature over the unchanged implementation domain. K.CT.LazyExtendedStandard assumes the restricted all-singleton LazySerialScope, an accepting lazy Policy-S witness with complete LazyClosure and LazyRank hypotheses, and the protected extension. The original issue/admission ranks remain valid when the added actual completions produce standard closure, allowing K.CT.Standard to apply. A local singleton extension in a mixed design instead feeds LazyComposition with its own complete assembled-graph acyclicity evidence; extension eligibility alone does not establish the restricted corollary. Extending the source witness after R imposes no RTL service after R: the completion induction uses the hypothetical execution that never accepts R, where service remains in force.

For the demand-supported branch, K.CT.RecordedDemand binds the source occurrence, issue/batch evidence and complete prospective control/preparation support. DemandPlacement extends the alignment/placement target domain to such a supported singleton, without fabricating its completion. BatchPlacement establishes arrival-independent placement and legal requests for the entire qualified input batch without assuming missing sibling admissions; an admission genuinely needed for control or descriptor preparation remains a predecessor.

K.CT.DemandPushResource proves persistent ingress space and credit for an extra-slack target in three cases: the supporting Pop has retired; its item is already outside ingress; or completed support establishes isolation and a pending request, so existing independent forwarding moves the oldest item out. Endpoint order prevents a later Push stealing the released space. The argument uses existing isolation-scoped progress, not bubble compression before isolation. Only the ingress-space/credit component of the designated extra-slack case changes: combine it with the target Push’s placement, request, guard proof and every other resource conjunct. Ordinary Push cases retain the standard resource lemma.

Auxiliary demand nodes are removed by replacing their support-to-demand-to-target paths with support dependencies. W contains actual completions only. For an atomic member’s extra-slack release, use member-demand support even if the group completes later in the witness; genuine sibling admissions remain

dependencies of the group completion unit. Extension-completed singleton floor edges can coexist with member demands under LazyMixedRank. No source demand record asserts an RTL request or completion.

Theorem K.CT.LazyComposition combines selected eligible channels with other qualified components in one finite complete certificate. Each unit has its applicable Exposure proof under the predecessors in that same graph and over all admitted surrounding activity. Acyclicity must be proved or soundly checked after demand elimination. A topological sort proves only acyclicity, not completeness, actual witness membership or validity of exposure. Corollary K.CT.LazySerialRestricted supplies the restricted globally serial/group-serial specialization, where the rank lemmas derive acyclicity. K.CT.LazyExtendedStandard remains the separate restricted all-singleton protected-extension return to Standard. All routes conclude through the unchanged K.CT.Completion theorem.

Completion route	Resource treatment	Acyclicity evidence
K.CT.Standard	Actual completed floor releases and all other standard obligations.	Complete standard closure with acyclicity.
K.CT.LazyExtendedStandard	Restricted all-singleton protected extension adds missing actual Pop completions.	Complete LazyClosure and original LazyRank hypotheses; resulting standard closure.
K.CT.LazySerialRestricted	Demand support only for qualified extra-slack cases; ordinary Pushes retain floor resources.	Global serial/group-serial construction with LazyRank or LazyMixedRank.
K.CT.LazyComposition	Selected local lazy cases compose with other qualified unit proofs.	Proof or sound check of the complete assembled graph after auxiliary-demand elimination.

Exposure, service and the theorem

K.CT.Exposure says that, for any incomplete W unit whose predecessors are complete, every admitted service-qualified continuation either completes it after finitely many steps or reaches continuous actual enabling until it completes. Atomic and rendezvous units require simultaneous whole-unit enabling. Persistent requests alone are insufficient if resources or guards keep disappearing. A general-theorem application using a multi-member residual unit must separately prove that its residual enabling and service complete the whole unit from every reachable already-retired subset. One member receiving service is insufficient. K.CT.Standard instead uses singleton partial-stage units.

K.CT.Service distinguishes obtaining an opportunity from committing an actual handshake. Core.TransferLaw and Core.SyncProtocol supply physical commitment on the eligible settled edge, with the required clocks and support maintained to that edge. Where arbitration is needed, B2 applies to independently defined pre-grant eligibility that excludes the arbiter’s own grant or denial; the proof also binds the grant to the required support or transfer. A finite eventual grant can directly establish Exposure’s finite-completion branch. Abstract units, preparation, computation, request assertion, internal movement and restoration retain their declared service laws. A coupled unit needs a whole-unit argument; separate endpoint fairness is insufficient. Intermittent eligibility needs its own qualified progress contract, not an implicit global strong-fairness assumption. Each service contract names an actual action/opportunity and independently justified support and execution

conditions. Qualified recurrent-opportunity and bounded-response contracts remain possible. A contract cannot merely assume R, eventual required outputs or the unit-completion conclusion it is being used to prove; “until R” states the obligation’s duration, not that R occurs. B3’s conditional primitive-channel progress is independent of B2, and neither is a blanket prerequisite for finite-prefix witness selection.

At an ordinary uncoupled FIFO, once the obstruction clears, a still-persistent request commits at the next eligible settled edge, assuming the edge occurs without an affecting reset. No B2 fairness is needed for that already enabled transfer. Old-state FIFO tests still prohibit same-edge fall-through and full-FIFO replacement. Coupled boundaries may remain blocked by other inputs, resources or grants, so their Q-level exposure and progress obligations remain. These facts do not prove that the obstruction clears, a peer arrives or a future edge occurs.

K.CT.Completion is a finite topological induction over W. Once predecessors complete, Exposure and Service complete the next unit. Facts and Agreement identify its occurrence, value and endpoint order. Thus all required acceptances occur. Service may be required only through R; completing additional W members beyond that acceptance point requires the stronger service scope stated in K. The theorem gives no uniform time bound.

K.CT.Standard discharges Exposure for the declared B.OF/B.CT/Q.CT profile. It first uses K.CT.Alignment for both the target’s descriptor and every CtrlSupport decision, including negative membership and entry decisions: agreed historical facts determine the source-required operations, and B.OF.Control/B.CT.Control bind those decisions to actual finite legal controller preparation, with B.OF.Control supplying the source-aligned pipeline membership and iteration rules. PlacementLemma then uses the aligned decisions to show that older iterations and earlier stages contain no unwitnessed required work, and derives physical placement and eventual permanent isolation after that older work exits. Request, resource, residual/joint and synchronization lemmas establish persistent enabling. The controller cannot make an obligation vacuous by simply omitting the source-required operation. The lazy route reuses these control and progress foundations. DemandPlacement generalizes the supported target to a recorded singleton demand outside W; BatchPlacement supplies the qualified resource-independent whole-batch preparation step. DemandPushResource replaces designated extra-slack resource cases only. LazyComposition combines those cases with retained standard proofs rather than weakening standard closure globally.

Why the standard floor route remains useful

B.CT.FIFO distinguishes nominal modeled storage from guaranteed usable space. With an exclusive ingress queue of F entries and optional outstanding credit C, a positive guaranteed floor g satisfies $g \le F$ and, when applicable, $g \le C$. Reserved slots and downstream pipeline storage do not automatically enlarge this floor.

For a standard initially empty positive resource, use B.CT.EndpointOrder’s zero-based, epoch-qualified endpoint ordinals $k \in \{0, 1, 2, \ldots\}$. W includes Push(k) before Pop(k), and Pop(k−g) before Push(k) whenever $k \ge g$. Those space edges apply even when the witness used the full nominal reference capacity B. After adding them, the graph must still be acyclic. For example, nominal capacity four with a justified floor two requires Pop(0) before Push(2) and Pop(1) before Push(3); a source pass relying on all four nominal slots does not waive those prerequisites. The boundary case k = g is the edge most easily lost: the standard floor proof for g=F requires Pop(0)’s retirement to establish space for Push(g). Eligible lazy demand support is a different sufficient proof that can establish space before that retirement.

Running the witness at a smaller capacity is optional and additionally requires K.CT.Capacity’s legal lift into the fixed reference, supported on both source and implementation sides. The safety model keeps the actual staged structure. An unsuccessful conservative witness or cyclic sufficient prerequisite graph is inconclusive, not a deadlock verdict. B=0 uses joint rendezvous semantics, never substitution into positive FIFO floor

arithmetic. Prefer lazy treatment where eligible, but keep the standard route for other components, physical movement without optional selection, or an already suitable standard certificate. It also supplies ordinary Push cases within LazyComposition and the restricted all-singleton LazyExtendedStandard result. A floor is a reusable resource guarantee, not necessarily the capacity used to run the simulation. A failed lazy witness is likewise inconclusive; another qualified witness or route may succeed.

9 Qualifying pipeline and resource progress

B.OF applies to the declared ordered finite-stage no-wait pipeline profile. It preserves iteration order and exact retirement state, requires serviced finite local computation, and ensures completed downstream work can drain independently of blocked upstream work or new input. Bubble compression is permitted but not required: an empty location behind incomplete downstream work does not by itself force advance. An unconditional global stall that indefinitely freezes required completed downstream work fails this profile.

B.CT adds source-aligned controller behavior, exact storage ownership and endpoint order, capture and guard laws, atomic compatibility, synchronous update semantics, synchronization/restoration and service. Its laws apply over all admitted reachable states and surrounding activity, including activity outside W. A no-wait body, synthesis directive or B-faithfulness alone does not establish them.

Resource organization	Named progress qualification	Distinction to preserve
Queue with capture at the Pop site	B.CT.FIFO and B.CT.Capture	Split capture retains the item; coincident retirement uses the old head
Independent early input paths	Q.CT.InputStaging and InputStagingProgress; eligible lazy cases use K.CT.DemandPushResource	Independent isolated delivery remains required. Lazy treatment uses source demand support for extra slack; ordinary cases retain the floor proof
Equal-length coupled positive pair	Q.CT.CoupledInputStaging and K.CT.CoupledDeliveryLemma	Joint support and clocks; mandatory physical movement or separately qualified abstract/internal service
Fixed-offset late second read	Q.CT.StaggeredInputStaging and K.CT.StaggeredDeliveryLemma	Track the early-only prefix, matched late read and paired suffix separately
Synchronization-gated regions	B.CT.Sync and Restore; Q.CT.Epoch; K.CT.SyncLemma	Restoration cannot depend circularly on traffic through the closed gate
Observation-neutral logical retry	Q.CT.Retry	Attempts implement one qualified persistent obligation under its own resource scope

B.BoundaryBinding distinguishes physical admission/movement, process-facing Issue and retirement, and legal source launch. Accepted unmatched input storage belongs in the channel accounting even when physically inside a pipeline. Registers after the selected Pop are process state. The last source-ordered message Issue of iteration j cannot follow the first message Issue of iteration j+1. For a regular unstalled schedule with fixed issue offsets, last_offset − first_offset ≤ II. This constrains source-facing Issue, not physical stage numbers;

stalls, delayed assertion, legal groups and the selected boundary map still require the actual E3 proof. Deep physical read offsets do not excuse that constraint; a qualified early-storage interpretation may be needed.

For the standard staging delivery proofs, target placement, legal preparation and persistent operation-owned request assertion must be established without assuming the target item's arrival at the terminal location or its retirement. Earlier forwarding is permitted, but the delivery conclusion cannot discharge its own placement/request premises, and an internal pending bit cannot be substituted for a gated physical request without the Core.10 binding and exact transfer proof. GQ2 may make the gated level the request, but it cannot turn arrival-dependent issue into an arrival-independent staging premise. A controller whose only route to terminal placement and its request is travel with the item needs the separately proved Q.CT.Extension pre-request route. A data-travelling valid bit alone does not decide applicability; the question is whether the placement/request proof depends on the arrival it is used to establish.

B.CT.Guard allows immediate enabling under persistent support, or a finite-response realization with an actual internally serviced, non-replenishable decreasing measure. Joint units need the joint guard proof. Once true, the guard remains true while its support and the request persist, until retirement. Persistent support is a separate conclusion: PlacementLemma supplies persistent isolation, and the applicable ResourceLemma, InputStagingProgress or delivery proof supplies persistent resources; the guard law establishes neither. These guards establish readiness before overlap; they cannot veto an actual settled handshake. A generic assertion that the transfer will eventually happen would merely assume the result the exposure proof needs.

Sync/epoch qualification preserves resident items, credits and global matching order. A fully drained boundary is one sufficient case, not a universal requirement. Gate restoration must be noncircular, and the protected target cannot lose availability to a later withdrawal even before its request becomes pending. ClockRefinement is an optional exact interleaving presentation for its narrower canonical capture/guard/advance profile; matching finite edges alone does not transport progress.

The named input-path alternatives are not freely interchangeable. Computed representation does not supply delivery; coupled paths need joint movement rather than separate per-input fairness. Fixed-offset staging does not qualify variable offsets, arbitrary unequal paths, staggered process-facing retirements or unconditional global stall. Mixed positive/B(0), retry and epoch-staging combinations retain their explicit compatibility requirements. A constructive realization that uses simultaneous-replacement staging also discharges Q.CT.InputStagingResidue or an explicitly mapped equivalent C3 argument. Completion service establishes no sustained initiation interval or throughput rate.

K.CT's "Explicit exclusions and extensions" gives the standard-profile list: "Standalone non-retry NB attempts as W units, multi-member non-atomic residual units, early capture outside the applicable Q.CT.InputStaging, Q.CT.CoupledInputStaging or Q.CT.StaggeredInputStaging clauses (whether single- or multi-member), arrival-dependent terminal placement/request assertion without the separately proved Q.CT.Extension pre-request route, recursive readiness networks, shared credit pools, arbitration outside the explicitly qualified pre-grant B2, B.OF.Service internal-arbiter or B.CT.Guard realization contracts, timing-dependent NB/signal control, resets crossing outstanding obligations, and synchronization among more than two participants require additional stated laws and proofs." These exclusions do not exclude otherwise qualified NB observations from safety; a consumed NB predecessor still needs its own completion discharge. Q.CT.Extension is the explicit route for those additional laws and proofs. Failure to fit the standard profile does not prove the design incorrect, and safety eligibility of a feature does not establish its completion eligibility.

Local eligibility in a mixed design

K.CT.LazyEligibility is local to selected independent InputStaging channels. They have empty-start exclusive ordered accounting, the declared ingress/credit law, and no added producer epoch gate or gated internal

movement. Their consumers use serial or qualified group-serial issue and satisfy DemandPlacement or BatchPlacement, including complete control/preparation support and arrival-independent request assertion. An incompatible overlapped consumer or data-driven pre-request placement does not inherit these lemmas. Producer issue need not be serial for the local space/credit argument, but its source legality, endpoint order, placement, preparation and exposure still require proof. The restricted rank corollary has stronger global issue premises.

If the selected demand is an atomic member, eligibility covers the whole group: pure positive, compatible same-site placement, complete independent batch preparation, and each input independently staged or an unreserved FIFO retiring by coincident capture on the commit edge. A plain FIFO here has no attributed slot. Split capture, a B(0) leg or a coupled/staggered pair in that batch prevents this demand treatment. These components can occur elsewhere in a LazyComposition design with their own standard or separately qualified proofs. Missing floor completions of unselected components remain missing obligations; composition does not manufacture them.

Selected channels and their certificate are fixed independently of the RTL execution being proved. Selection may follow source simulation only when the entire recorded trace, including any extension, satisfies the lazy restriction on every selected channel. Generating lazily on all eligible channels and choosing a subset afterwards is permitted, provided nonselection was legal. Gates elsewhere retain restoration/persistence proofs; a selected gated channel needs a separately proved combination not supplied by this first profile.

Ordinary synchronization does not reset global channel ordinals within the accounting epoch, and the consumer’s preceding blocking demand retires before its own sync completes. Other owners may synchronize while it remains pending under their contracts. Reset, cancellation and capacity withdrawal still require their actual preservation qualifications. Mixed singleton source extension retains the protected continuation window over the whole bound design; it does not prove arbitrary peer or group completion. Local extension eligibility establishes neither global LazyClosure nor the rank hypotheses of LazyExtendedStandard; retain the complete mixed graph and its separate acyclicity evidence.

10 Snooping and latency-sensitive choices

Appendix L remains relevant when hidden outcomes affect later visible behavior or attribution. L.WC records witness-compatible ε choices, including failed-call identities used by E3/E5. NB-CF, snooping, direct construction and a complete compatible per-site mixture are ways to establish the relevant facts. The direct constructive route derives outcomes and call attribution from local contracts and matched snapshots; it does not automatically require an additional Appendix L package.

A mixed per-site discharge needs a complete partition within one bound instance and one joint construction. Selected outcomes must be simultaneously legal under the common settled snapshot and source decision law, with compatible choices, footprints, dependencies and atomicity. Independently proved per-site outcomes do not establish that joint legality.

L.JointOptionality is a separate set-level proof for omitting designated snoops: all admitted joint variations must preserve the selected canonical DUT history at independently matched logical cuts, with separate legal witness/cut availability and coverage. Optionality at each site while all other sites remain snooped is insufficient. A class claim requires the proof uniformly over the class. Boundary-only optionality does not erase wider proof-internal or internal-testbench matching obligations, and it waives no K premise.

Snooping must select a legal source behavior. It cannot override Core.NBDecision, change fixed stimulus, suppress a mandatory physical transfer or sync completion, manufacture an NB failure, substitute payloads or bypass source phase restrictions. Cadence-sensitive retry counts, timeouts or arbitration need the qualified

latency-sensitive protocol/alignment treatment; calling failures ε does not erase their effects on later behavior. Snooping cannot add a missing source constraint or enforce a causal dependency absent from the source model. Properties depending on a particular order must model or justify it; a favorable replay is not a universal source-property proof. Fairness applies only when a selected consumer requires it.

When a snooping harness is used, W1–W5 require recorded per-ordinal observations, source-side mirror gating, qualified registered sampling, exact identity/kind/payload checks, outstanding-event accounting and complete choice coverage, consistently with Core.TransferLaw and Core.SyncProtocol. Mirror gating needs a qualified boundary/request binding and must preserve every issued request's lifecycle. L.Dir keeps the RTL free-running and requires ordered source consumption of its recorded observations. A live AND of current signals can lose or alias events when the source lags. An unproved watchdog bound expiring is an inconclusive witness-realization failure, not a proved design deadlock. A finite clean log does not prove future consumption or universal correspondence.

K has no standalone snooping premise. It consumes the selected J adapter with all that adapter's applicable premises, plus K.CT.Agreement and progress. Aligning one source witness with one observed RTL run does not prove agreement between a serial workload witness and every RTL-matched source prefix. A separately proved alignment may establish the required agreement, but recording outcomes alone does not do so. Snooping therefore neither replaces K's agreement nor proves exposure and service. K.CT.LazySimulation.RTLIndependence introduces no per-stimulus RTL run or probe: lazy selection observes only source-side demands. Structural implementation binding and reusable conformance still precede application. If a chosen fact/outcome qualification independently relies on snooping or co-simulation, it retains that dependency; a successful RTL run is not the lazy resource proof.

11 Applying and reviewing the framework

Start by declaring the intended claim: a safety predicate and observation, a required acceptance workload, or both. Fix the source/implementation artifacts, environment placement, choices, clock/reset, capacities and elaboration. Choose an actual completion carrier and feature profile. A narrowed observation can simplify a proof only when the property remains expressible and whole-group/order obligations are preserved.

For safety, establish source well-formedness and boundary/memory realization, then local process and primitive evidence, integration, operational settling/matching, environment and carrier coverage. Use ordinary or elaborated locality where its restrictions fit; otherwise discharge the broader named constructive contracts. Independently verify the actual configured source property on Reach_ref or qualified ChkRef, or establish the covering-execution alternative with its full premises. Bind monitor behavior to that same history semantics.

For completion, add the full fact signature and exact fact-prefix correspondence; provenance/agreement including the workload witness; finite acceptance witness and a complete acyclic certificate for the selected route, including K.CT.Closure and K.CT.CtrlSupport entry, branch and membership decisions (negative ones too) for the standard route; source-aligned controller and preparation evidence under the applicable B.OF.Control/B.CT.Control laws or the selected extension's proved replacement obligations; arrival-independent placement/request evidence for the standard staging proof, or the separately qualified pre-request delivery/placement argument under Q.CT.Extension; standard resource floors or eligible demand-supported extra-slack releases on zero-based ordinals, delivery, guards and atomic/sync compatibility; and mandatory physical commitment together with the required clock, environment, pre-grant and abstract/internal service. A source pass is only one input to this chain. Optional drained-test diagnostics do not replace acceptance or prove progress: empty monitored FIFOs alone do not establish a complete empty-inventory observation. Assess LazySerial eligibility first and prefer its source-only lazy workflow where it fits. Validate the recorded trace and complete support inventory; use LazyRank/LazyMixedRank in their restricted

domains or check the complete mixed graph under LazyComposition. A passing unrestricted Policy S run with exact capacities alone is not sufficient.

M.7a is the authoritative side-condition ledger. Each consumed premise has an owner, evidence, validity domain, trigger and invalidation rule, or a proof that its trigger is false. Framework-derived lemmas are not extra assumptions. One reusable artifact can discharge several named owners, but their obligations remain distinct. Tool/runtime qualification binds relevant versions, libraries, scheduling and reset/X semantics; unverified assurances belong in the trusted base. K's O1–O14 list is a progress-navigation checklist, not the complete premise list: retain Instance/Facts/Agreement, applicable sync/epoch/restoration contracts and Q.CT.Retry. M.7a now explicitly records logical-retry persistence, attempt mapping, no item/credit consumption or completion on failure, retained guard progress, admitted resources and required service. Tool and library examples are informative descriptions; verify the version and configuration in use, and distinguish normative tool requirements from evidence that a tool meets them.

For lazy applications, M.7a separates common, restricted, mixed-composition and simulator/extension evidence. Common records include selected-channel and whole-batch eligibility, source trace compliance, demand identities and CtrlSupport, ordinary versus extra-slack Push classification, the applicable graph/acyclicity discharge and each local exposure proof. Restricted applications record global serial/group-serial discipline and the prescribed complete closure/rank construction. Mixed applications record local selected-channel consumer discipline, the qualified modes of other owners and one complete assembled graph, without importing restricted global issue or rank requirements. A protected singleton extension additionally records the admitted item, retired lower Pops, complete preparation and whole-design continuation protection. Record added fact coverage and Agreement for the actual witness, and distinguish simulator demand-visibility/selection evidence from any independently chosen snooping dependency. None of this requires observing RTL progress for each stimulus.

Evidence level	Typical responsibility
Framework	Prove generic construction, projection, agreement and workload induction; lazy inventory/slack, demand placement/resources, auxiliary-node elimination, restricted ranks, protected extension and route assembly
Tool or component family	Prove qualified transformations, primitives and controller/runtime laws; for lazy simulation qualify arrival-independent source demand visibility, legal selection, old-state updates and mandatory transfers
Design or stimulus class	Supply local process invariants, integration/environment evidence and independent source-property proofs
Concrete application	Bind artifacts, topology, boundaries, workload and residual evidence; for lazy treatment include eligibility, source trace/demand support, restricted rank validation or mixed-graph acyclicity, and any protected extension

Core.EvidenceRealization distinguishes a mathematical proof family, a sound total extractor on a qualified domain, and individually checked outputs of a partial generator. A partial generator's failures cannot be excluded from a universal implementation claim merely because no certificate was produced. Core.ExtensionConsumerCoverage requires every admitted extension to have each applicable consumer case or a proved inapplicability; a safety extension with no fact or progress case is not automatically completion-qualified.

A useful review follows the dependencies rather than counting checks. Ask whether source legality is independent of construction success; whether every actual successor is covered; whether all consumed observations and facts are present; whether property validity is separate from transformation correctness; and whether each progress step establishes the support and clocks needed for mandatory commitment or the separately qualified service of an abstract action. Reuse remains valid only while its semantic and artifact bindings remain within the proved domain. The current rules specify these proof interfaces; they do not claim a completed universal tool qualification or an unconditional verification-cost advantage. A clean finite runtime assertion log is exercised-prefix evidence, not a proof of a universal no-backpressure invariant or class-wide conformance. For lazy checking, a topological sort establishes no inventory completeness or exposure validity, and prompt deterministic selection guarantees neither schedule dominance nor that the generator finds every qualifying witness. The source witness's serial discipline does not impose serial RTL scheduling.

12 Where to read next

Question	Current rules location
Which user design constraints matter	Appendix F and the main scheduling rules
What models and source steps are legal	Normative Formal Core, especially Core.10, Core.TransferLaw, Core.SyncProtocol and Core.Phase; Appendices B, C and G
How local evidence produces global safety	J.LocalOriginContracts; J.ConstructiveEdgeInduction; J.LG-Origin; J.BatchPrefixCoverage
When composition is genuinely local	J.OrdinaryConstructiveComposition and J.ElaboratedConstructiveComposition
How source safety and class coverage are discharged	J.SafeTransfer-HistCovOn; source-property monitor contract; J.ProcessEnvClass; UCL0–UCL3
How internal facts connect to workload completion	J.2c, including J.StableReadFactCorr and J.ConfigRecordFactCorr; Core.FinitePrefixProvenance and Core.FixedSupportValidation; K.CT.Instance through K.CT.Completion
How the standard exposure proof works	B.OF.Control/B.CT.Control; B.OF and B.CT progress laws; K.CT.CtrlSupport, Alignment and Standard; Q.CT and the Q.CT.Extension pre-request route; Appendix N
How choices and snooping are qualified	Appendix L and the L.WC/L.Dir entries in M.7a
How elaborated boundaries and accounting are bound	Core.ElabProj and Appendix Q.1
What evidence and mechanization remain	M.7/M.7a; Appendices R, S and T

Question	Current rules location
How lazy pre-HLS simulation works	§8 workflow and singleton/atomic examples; K.CT.LazySerialWitness; K.CT.LazySimulation and its DemandVisibility, Selection, Example and RTLIndependence clauses
How lazy completion composes with other components	§8 route comparison; K.CT.LazyEligibility; LazySourceExtension; DemandPlacement and BatchPlacement; DemandPushResource; Theorem LazyComposition; Corollary LazySerialRestricted; restricted all-singleton LazyExtendedStandard; route-specific M.7a evidence

The controlling dependency is consistent throughout: legal local source realization establishes correspondence; an independent source proof establishes safety; full facts and agreement identify the workload's required operations; concrete exposure and service make those operations complete. Each application must preserve those boundaries while instantiating the parts it actually consumes.

Revision note — 26 October 2026. Added the preferred source-only lazy simulation workflow, demand visibility and staging example, local eligibility, finite support and rank construction, protected singleton extension and demand-resource branch. Integrated the LazyComposition theorem and LazySerialRestricted corollary, mixed-design boundaries, Facts/Agreement and CtrlSupport coverage, retained standard-floor uses, M.7a evidence and named simulator guidance. The guide remains informative and asserts no new implementation qualification or completed mechanization. Follow-up integration aligns the opening guidance, issue/movement/timing distinction, start-message obstruction, atomic-batch example, route comparison and evidence responsibilities with the lazy-integrated rules. LazyExtendedStandard is explicitly restricted to the all-singleton construction; mixed extensions retain independent assembled-graph evidence, and demand replacement preserves guards and other resource conjuncts.

Revision note — 25 October 2026. Aligned the attached 23 October guide with the rules cross-audit follow-up: clarified evaluated-boundary request persistence, strict synchronization timing, mandatory capacity withdrawal versus old-use isolation, reset/termination observations, reactive testbench binding, Core.FixedSupportValidation, completion-domain quantification, mixed-owner closure, residual-unit and noncircular service obligations, qualified arbitration, and retry/evidence checklist scope. The earlier medium-repair alignment is retained. The rules remain authoritative; this guide does not close the baseline audit or certify a concrete implementation.

Revision note — 23 October 2026. Aligned this guide with the rules doc: removed the cycle-ideal application route; reconciled issue latitude, complete physical transfer inventories, pre-grant fairness and primitive progress; added gated-request and sync lifecycle explanations; and updated the related construction, staging, snooping and evidence guidance. The rules doc Appendix U consolidates the history.